

** Batalla Naval Distribuido con Java RMI**

Introducción

Este proyecto presenta un **juego de Batalla Naval distribuido** que utiliza **Java RMI (Remote Method Invocation)** para permitir que dos jugadores compitan desde computadoras diferentes conectadas a la misma red local. El sistema demuestra conceptos fundamentales de **sistemas distribuidos** aplicados de manera práctica y funcional.

Arquitectura del Sistema Distribuido

Modelo Cliente-Servidor con RMI

El proyecto implementa una arquitectura **cliente-servidor distribuida** donde:

- **Servidor RMI:** Coordina el juego, mantiene el estado y arbitra las reglas
- **Clientes RMI:** Proporcionan interfaz gráfica y se comunican con el servidor
- **Componentes Compartidos:** Definen el "lenguaje común" entre cliente y servidor

Estructura Modular del Proyecto

El sistema está organizado en **3 módulos Maven** independientes:

1. **shared** - Define los contratos y modelos de datos compartidos
2. **server** - Implementa la lógica del servidor de juego distribuido
3. **client** - Proporciona la interfaz gráfica y cliente RMI

Esta separación permite **desarrollo independiente, despliegue flexible y mantenimiento simplificado**.

Detalle de Módulos

1. Módulo **shared** - Componentes Distribuidos

```
shared/
├── pom.xml
└── src/main/java/co/edu/uptc/shared/
    ├── interfaces/           # Interfaces RMI
    │   ├── GameCallback.java # Callbacks para notificaciones
    │   └── GameService.java  # Servicio principal RMI
    └── model/                # Modelos de datos serializables
        ├── Board.java        # Tablero de juego
        ├── GameStats.java    # Estadísticas de partida
        ├── GameStatus.java   # Estado del juego
        ├── PlayerStats.java  # Estadísticas del jugador
        ├── Position.java     # Coordenadas inmutables
        └── Ship.java         # Representación de barcos
```

Propósito: Define el contrato distribuido entre cliente y servidor, incluyendo interfaces RMI y modelos de datos serializables que viajan por la red.

2. Módulo **server** - Servidor Distribuido

```
server/
├── pom.xml
└── src/main/java/co/edu/uptc/server/
    ├── GameServiceImpl.java    # Implementación del servicio RMI
    ├── GameSession.java       # Gestión de sesiones concurrentes
    ├── Player.java             # Entidad jugador distribuida
    └── ServerMain.java         # Bootstrap del servidor RMI
```

Propósito: Núcleo del sistema distribuido que gestiona múltiples sesiones, coordina clientes y mantiene la consistencia del estado de juego.

3. Módulo **client** - Cliente GUI Distribuido

```
client/
├── pom.xml
└── src/main/java/co/edu/uptc/client/
    ├── BoardPanel.java        # Panel del tablero interactivo
    ├── ClientMain.java        # Punto de entrada del cliente
    ├── GameCallbackImpl.java  # Implementación de callbacks RMI
    ├── GameController.java    # Controlador con comunicación RMI
    ├── GameStatsPanel.java    # Panel de estadísticas
    ├── GameWindow.java        # Ventana principal GUI
    └── NetworkDiscovery.java   # Auto-descubrimiento de servidores
```

Propósito: Interfaz gráfica rica que actúa como cliente RMI con capacidades de comunicación bidireccional y tolerancia a fallos.

Funcionalidades RMI Implementadas

Comunicación Distribuida Bidireccional

Del Cliente al Servidor (Invocaciones RMI tradicionales):

- **joinGame(String playerName, GameCallback callback):** Conectarse al juego
- **placeShip(String playerId, Position start, Position end):** Colocar barcos
- **attack(String playerId, Position target):** Realizar ataques
- **getGameStatus(String playerId):** Consultar estado del juego
- **heartbeat(String playerId):** Mantener conexión activa

Del Servidor al Cliente (Callbacks RMI):

- **onTurnChanged(boolean isMyTurn, String currentPlayerName)**: Notificar cambios de turno
- **onAttackEvent(String attacker, Position target, String result)**: Informar resultados de ataques
- **onGameEnded(String winner)**: Anunciar fin del juego
- **onOpponentDisconnected()**: Alertar desconexiones
- **onGameEvent(String message)**: Mensajes generales del juego

Gestión Inteligente de Conexiones

El sistema implementa **múltiples mecanismos** para mantener la conexión y recuperarse de fallos:

1. **Sistema de Heartbeat**: Detecta automáticamente cuando un jugador se desconecta (cada 15-30 segundos)
2. **Callbacks + Polling**: Si fallan las notificaciones inmediatas, usa consultas periódicas como respaldo
3. **Reconexión Automática**: Intenta recuperar conexiones perdidas con backoff exponencial
4. **Cleanup Automático**: Libera recursos de jugadores desconectados

Experiencia de Usuario Distribuida

Flujo de Juego Multiplayer

1. **Descubrimiento Automático**: El cliente busca servidores disponibles en la red local
2. **Conexión Transparente**: Se conecta al servidor sin configuración manual de IPs
3. **Sincronización en Tiempo Real**: Los movimientos de un jugador aparecen instantáneamente en el oponente
4. **Manejo de Desconexiones**: Si un jugador se desconecta, el sistema lo detecta y notifica al oponente
5. **Estadísticas Finales**: Panel detallado con métricas de la partida

Características de la Interfaz Distribuida

- **Dos Tableros Visibles**: Tu tablero (donde colocas barcos) y tablero del oponente (donde atacas)
- **Indicadores de Turno**: Muestra claramente quién debe jugar en cada momento
- **Mensajes en Tiempo Real**: Notificaciones instantáneas de eventos del juego
- **Panel de Estadísticas**: Métricas detalladas durante y después de la partida
- **Indicador de Conexión**: Estado de la conexión con el servidor

Aspectos Técnicos de la Distribución

Manejo del Estado Distribuido

Desafío: Mantener consistente el estado del juego entre múltiples computadoras conectadas por red.

Solución Implementada:

- El **servidor mantiene la autoridad única** del estado del juego (single source of truth)
- Los **clientes son "vistas sincronizadas"** de ese estado central
- **Sincronización automática inmediata** via callbacks cuando algo cambia
- **Validación server-side** de todas las operaciones para prevenir inconsistencias

Tolerancia a Fallos de Red

Problemas Típicos en Redes Distribuidas:

- Conexiones que se pierden temporalmente (WiFi inestable)
- Latencia variable entre computadoras
- Jugadores que cierran la aplicación inesperadamente
- Paquetes perdidos o retrasados

Soluciones Implementadas:

- **Detección Proactiva:** Sistema de heartbeat que detecta problemas antes de que afecten la experiencia
- **Recuperación Automática:** Reconexión transparente para fallos temporales de red
- **Mecanismo de Respaldo:** Si fallan los callbacks, el polling mantiene la sincronización
- **Notificaciones Informativas:** El usuario siempre sabe qué está pasando con su conexión

Concurrencia y Sesiones Distribuidas

Gestión de Múltiples Partidas:

- **Sesiones Aisladas:** Cada partida mantiene estado independiente (`GameSession`)
- **Thread Safety:** Uso de `ConcurrentHashMap` y sincronización para operaciones concurrentes
- **Balanceador Automático:** Distribución inteligente de jugadores entre sesiones disponibles
- **Cleanup Inteligente:** Liberación automática de recursos cuando las partidas terminan

Seguridad y Restricciones

- **Solo Redes Locales:** El sistema solo acepta conexiones desde rangos IP privados (192.168.x.x, 10.x.x.x)
- **Validación Server-Side:** Todas las jugadas se validan en el servidor
- **Gestión de Recursos:** Liberación automática de memoria cuando los jugadores se van
- **Control de Acceso:** Verificación del origen de las conexiones para seguridad

Despliegue y Uso Práctico

Facilidad de Distribución

El proyecto incluye un **paquete completo** para usuarios finales en `paquete_cliente_completo/`:

- **battleship-client-jar-with-dependencies.jar:** Ejecutable todo-en-uno
- **INICIAR.bat:** Script de inicio simple para Windows
- **verificar.bat:** Verificador de sistema y dependencias
- **all.policy:** Políticas de seguridad RMI preconfiguradas

Proceso de Instalación Zero

Para jugar, los usuarios solo necesitan:

1. **Descargar** el paquete cliente completo
2. **Ejecutar** el script `INICIAR.bat`
3. El sistema **encuentra automáticamente** otros servidores en la red

4. **Seleccionar servidor** de la lista descubierta y empezar a jugar

Comandos de Despliegue

Servidor RMI:

```
java -Djava.security.policy=all.policy \  
-Djava.rmi.server.hostname=[HOST] \  
-jar server-1.0-SNAPSHOT.jar [puerto]
```

Cliente:

```
java -Djava.security.policy=all.policy \  
-jar battleship-client-jar-with-dependencies.jar [host] [puerto]
```

Beneficios de la Arquitectura RMI

Para el Desarrollo

1. **Simplicidad de Programación:** RMI oculta la complejidad de la programación de red de bajo nivel
2. **Transparencia de Ubicación:** Los métodos remotos se usan exactamente como métodos locales
3. **Serialización Automática:** Java maneja automáticamente el envío de objetos complejos por red
4. **Integración Natural:** Se integra perfectamente con el ecosistema y herramientas Java existentes
5. **Type Safety:** Verificación de tipos en tiempo de compilación para interfaces remotas

Para los Usuarios Finales

1. **Experiencia Fluida:** Respuesta prácticamente inmediata a todas las acciones
2. **Alta Confiabilidad:** El sistema se recupera automáticamente de problemas menores de red
3. **Configuración Mínima:** No requiere conocimientos técnicos o configuración compleja
4. **Rendimiento Optimizado:** Diseñado específicamente para redes locales (baja latencia)
5. **Feedback Continuo:** Información clara sobre el estado de conexión y juego

Conceptos de Sistemas Distribuidos Aplicados

1. **Comunicación Remota:** Implementación práctica de RPC (Remote Procedure Calls)
2. **Gestión de Estado Distribuido:** Técnicas para mantener consistencia entre múltiples nodos
3. **Tolerancia a Fallos:** Diseño de sistemas que funcionan aún con problemas de red
4. **Descubrimiento de Servicios:** Mecanismos para que clientes encuentren servidores automáticamente
5. **Concurrencia Distribuida:** Gestión de múltiples usuarios y sesiones simultáneas
6. **Serialización de Objetos:** Transferencia eficiente de datos complejos por red aislamiento |

Patrones de Diseño Distribuido Aplicados

1. **Remote Proxy Pattern:** Stubs RMI actúan como proxies de objetos remotos

2. **Observer Pattern Distribuido:** Callbacks para notificaciones asíncronas
 3. **Session Pattern:** Gestión de contexto y estado por sesión de usuario
 4. **Heartbeat Pattern:** Monitoreo proactivo de salud de conexiones
 5. **Registry Pattern:** RMI Registry como servicio de directorio distribuido
-